

The installed apps are listed in the `INSTALLED_APPS` constant in `setting.py`.

```
INSTALLED_APPS = (
    'rest_framework',
    'Task',
)
```

Any global settings for a REST framework API are kept in a single configuration dictionary named `REST_FRAMEWORK`.

```
REST_FRAMEWORK = {
    'DEFAULT_MODEL_SERIALIZER_CLASS':
        'rest_framework.serializers.
ModelSerializer',
}
```

Creating models

Let's go back to the Task app in the folder `Task` and create the models we need in the file `/Task/models.py`. In order to define a Task model, we need to derive from the Model class. Let's use `User` class of the standard authentication as a foreign key in the owner attribute.

We define an owner, task name, description of the task, a status and an attribute which stores the last time a model got updated.

```
from django.db import models
from django.contrib.auth.models import User
class TaskModel(models.Model):
    user = models.OneToOneField(User)
    task_name = models.CharField(max_length=100)
    task_description = models.TextField(max_length=200)
    status = models.BooleanField(default=False)
    date = models.DateTimeField(auto_now_add=True)
```

Initialising the database

We have to set up the database for storing data. In the default settings, an SQLite database with the required schema is automatically created with the following commands:

```
$ (env) python manage.py makemigrations
$ (env) python manage.py migrate
```

Creating serializers

We'll declare a serializer that we can use to serialise and deserialise data that corresponds to `TaskModel` objects.

Let's create a new module named `Task/serializers.py` that we'll use for our data representations.

The `ModelSerializer` class provides a shortcut that lets you automatically create a `Serializer` class with fields that correspond to the Model fields.

```
from django.contrib.auth.models import User
from rest_framework import serializers
from .models import TaskModel
class TaskSerializer(serializers.ModelSerializer):
    user = serializers.CharField(source='user.
username', read_only=True)
    class Meta:
        model = TaskModel
        fields = ('user', 'task_name', 'task_
description', 'status', 'date')
class RegistrationSerializer (serializers.ModelSerializer):
    password = serializers.CharField(write_only=True)
    def create(self, validated_data):
        user = User.objects.create(
            username = validated_data['username']
        )
        user.set_password(validated_data['password'])
        user.save()
        return user
    class Meta:
        model = User
        fields = ('username', 'password')
```

We defined two serializer classes `TaskSerializer` and `RegistrationSerializer`. `TaskSerializer` has the user as a `read_only` serializer field, which references our logged in user or we can say a user who has permission to perform CRUD operations on the Task model.

`RegistrationSerializer` has a password field as a `write_only` because we don't want to serialise the hashed password. We override the `serializers create()` method in order to create a user instance.

Adjusting the URL dispatcher

Now, let's set up the URL dispatcher in the file `/TaskAPI/urls.py`. The URL dispatcher sets the URL routes to specific views in Django. A view handles logic and sends HTTP responses.

The REST framework adds support for automatic URL routing to Django, using the `Routers.Django` REST framework, which provides `SimpleRouter`, `DefaultRouter` and `CustomRouters`.

Let's use `SimpleRouter` and URL patterns in this project for now.

We will implement `TaskView` for the creation, listing, deletion and updation of tasks and `RegistrationView` for user registration.

```
from django.conf.urls import include, url
from django.contrib import admin
from rest_framework import routers
from Task import views
```